

Scrapping articles tool

Samuel Bergina
samuelbergina@gmail.com

July 2024

1 Introduction

The scrapping article tool enable to scrape article content from a list of newspapers URL

The content that can be scraped

- Text
- Authors
- Date
- Images Urls, and alt
- Subheaders

2 Requirements

To use this program, you first need to install all the required libraries. You can find the list of these libraries in the requirements.txt file.

3 How to Use the Scraper for Articles That Don't Require a Subscription

Scrapping the Content of an Article

First, add a list of URLs from a single type of newspaper in the folder Data/Urlslist as a CSV file. The CSV file can contain multiple columns, but the program specifically needs a column named 'url'.

Next, go to the main function. At the last line of the code, specify the name of the newspaper you wish to scrape from and the type of content you want to scrape, such as text or images. You can enter these options as shown in the following example. The name need to be the same as the functions in the folder `Scrapping_articles/scrapping_specific`. Finally, launch the program by running main.py from the command line. The scraped content can be found in the folder:

Data/articles_scrapped

```
main('Spiegel', Author=False)
```

Note that the program only fetches the URL and downloads the HTML content from the webpage once. If the article you want to scrape has been updated, you need to delete the previously downloaded HTML content in the folder `Data/html_content` before scraping again.

Adding a New Newspaper or Updating the Scraper for a Specific Newspaper

If a newspaper's website is updated, the method used to select elements from the HTML content may need to be updated as well. To update the scraper: Navigate to the folder `Scrapping_articles/scrapping_specific` and select the newspaper you wish to update. Use a precise find method from BeautifulSoup to select the correct elements. You can get inspiration from the code for other newspapers. To add a new newspaper, for example, 'The Wall Street Journal', go to the folder `Scrappingarticles/scrappingspecific` and create a Python file with the following functions:

- `extract_subheadlines_from_html`
- `extract_author_from_html`
- `extract_text_from_html`
- `extract_date_from_html`
- `extract_image_from_html`

Follow the examples from other newspapers to implement these functions.

4 How to Use the Scraper for Articles That Require a Subscription

First, launch the script called `Save_session.py`. This script will open a Chrome browser. Navigate to the website of the newspaper and log in to your account. Ensure that you accept cookies to be saved. Once you are logged in, you can close the Chrome window. The cookies from this session will be stored and used to access the URLs listed in the list of URLs. This allows you to access all the subscribers' content.

You can now proceed to use the code as specified in Part Two. If you find that the scraped content is inaccessible because you are not seemingly logged in, please reconnect to the website accordingly.

Using the script on a server To use the `save_session` script, you first need to have a Chrome browser and the ability to use a graphical interface. Once you have that set up, you can launch the main program on a headless machine. If you want to run the scraping on a headless server, follow these steps: Launch the `save_session.py` script on your personal device. To use the session cookies saved in the `Data/Cookies_session` folder on the server, sync the local github repo with the server.